Algorithmen und Komplexität

Fortgeschrittene mathematische Methoden in der Statistik

Vorlesung: Fabian Scheipl (fabian.scheipl@lmu.de)
Material: Thomas Nagler
LMU München

Version: 15.10.2026

Verwendete Vorkenntnisse

- **Grundlegende Analysis** (aus Analysis I)
  - Folgen und Grenzwerte
  - Grundbegriffe der Konvergenz
- **Grundlegende Lineare Algebra** (aus Lineare Algebra I)
  - Matrixoperationen (Addition, Multiplikation)
  - Verständnis des Rechenaufwands von Operationen

Numerische Probleme und Algorithmen

Numerische Probleme

- Ein numerisches Problem hat als Lösung eine oder mehrere Zahlen.
  - Wert eines Integrals
  - Lösung eines LGS
  - Basis-Koeffizienten in Funktionsapproximation
- Allgemein:

> *Gegeben Problem f mit Input $\boldsymbol{x}$, berechne Lösung $f(\boldsymbol{x})$.*

Live-Demo: Rechnen Computer richtig?

Erst raten, dann ausführen!

Was geben diese Codezeilen aus?

```r
1.0 - 1.0
```

```
[1] 0
```

```r
1.0 - 0.9 - 0.1
```

```
[1] -2.78e-17
```

```r
100 * 0.58 == 58
```

```
[1] FALSE
```

```r
x <- seq(1, 2e16, length = 10^5)
sum(x) - sum(rev(x))
```

```
[1] -262144
```

Numerische Probleme: Schwierigkeiten

- Computer sind so gut wie nie exakt.
- Sie können
  - … nur endliche und endlich viele Zahlen darstellen.
    - $\pi = 3.14159265358979323846264338327 9\ldots$ 😞
  - … nur endlich viele Berechnungen durchführen.
    - $e^x = \sum_{n=0}^{\infty} \frac{x^n}{n!}$ 😞
- Deshalb ist es wichtig, *wie* wir Dinge berechnen!

Numerische Probleme: Schwierigkeiten

- Computer können nur endlich viele Zahlen darstellen.
- Das führt zu *Rundungsfehlern*, die sich *akkumulieren* können.
- Die *Reihenfolge* unserer Rechenoperationen spielt eine Rolle.
- *Fehler* einzelner Operationen sind oft vernachlässigbar, aber sie *akkumulieren*.
- **Empfehlung**: Besonders große/kleine Zahlen vermeiden.

Beispiel: Katastrophale Auslöschung

Problem: Berechne die Varianz von $\boldsymbol{x}$ mit der Formel

$$\mathrm{Var}(\boldsymbol{x}) = \frac{1}{n}\sum_{i=1}^{n} x_i^2 - \left(\frac{1}{n}\sum_{i=1}^{n} x_i\right)^2$$

Daten: $\boldsymbol{x} = (4 + 10^9,\ 7 + 10^9,\ 13 + 10^9,\ 16 + 10^9)$

Berechnung:

```
x <- c(4, 7, 13, 16) + 1e9
mean((x - mean(x))^2) # Varianz (mit 1/n)
```

```
[1] 22.5
```

```
mean(x^2) - mean(x)^2 # !! auch Varianz (mit 1/n) ?!
```

```
[1] 0
```

- **Ursache:** Subtraktion zweier fast gleich großer Zahlen ($\approx 10^{18}$).

```
mean(x^2)
```

```
[1] 1e+18
```

```
mean(x)^2
```

```
[1] 1e+18
```

Beispiel: Assoziativität verletzt

Problem: Gleitkomma-Addition ist nicht assoziativ!

Konkretes Beispiel: Seien $x = 10^{30}, y = -10^{30}, z = 1$.

$$(x + y) + z = (10^{30} + (-10^{30})) + 1 = 0 + 1 = 1$$
$$x + (y + z) = 10^{30} + ((-10^{30}) + 1) \overset{!!}{\approx} 10^{30} + (-10^{30}) = 0$$

- Die zweite Berechnung ergibt 0, weil 1 zu klein ist gegenüber 10^{30}.

$\Longrightarrow$ Reihenfolge der Summe beeinflusst Ergebnis!!

Algorithmen

Def.: Algorithmus

Ein **Algorithmus** ist ein Verfahren $\widetilde{f} = \widetilde{f}_s \circ \cdots \circ \widetilde{f}_1$, das für Inputs $\boldsymbol{x}$ eine mögliche Lösung $\widetilde{f}(\boldsymbol{x})$ des *Problems* $f(\boldsymbol{x})$ berechnet.

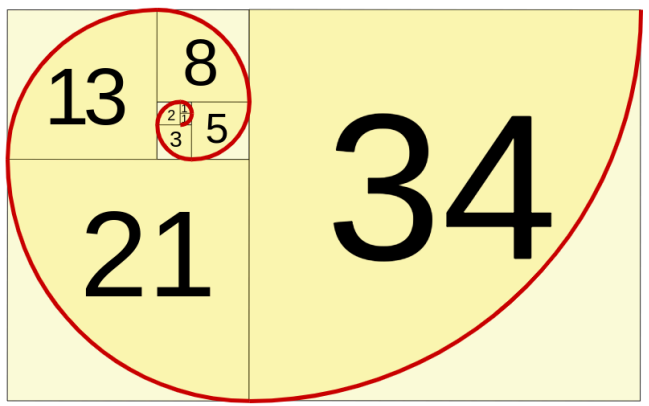
Arten von Algorithmen

- *exakt*: es gilt $f(\boldsymbol{x}) = \widetilde{f}(\boldsymbol{x})$ (bis auf Rundungsfehler)
  vs. *approximativ*: es gilt lediglich $f(\boldsymbol{x}) \approx \widetilde{f}(\boldsymbol{x})$.
- *iterativ*: gleiche Vorschrift wird mehrfach ausgeführt
  vs *direkt*: Vorschrift wird nur einmal ausgeführt.
- *probabilistisch/randomisiert*: beruht auf Zufall
  vs *deterministisch*: keine Zufallselemente.

Beispiel: Fibonacci-Zahlen

- Problem: Berechne die ersten N Fibonacci-Zahlen

$$x_1 = 0, \quad x_2 = 1, \quad x_{n+1} = x_n + x_{n-1}\ (n \geq 2).$$



- Algorithmus:

$$\widetilde{f} : \mathbb{N} \to \mathbb{N}^n$$
$$\widetilde{f}(n) = (\widetilde{f}_n \circ \cdots \circ \widetilde{f}_3 \circ \widetilde{f}_2 \circ \widetilde{f}_1)(n)$$
$$\widetilde{f}_1(n) = 0, \quad \widetilde{f}_2(x) = (x, 1),$$
$$\widetilde{f}_{i+1}(x_1, \ldots, x_i) = (x_1, \ldots, x_i, x_i + x_{i-1}) \quad (i \geq 2).$$

Fibonacci: Implementierung

- Implementierung des Algorithmus:

```
fibonacci <- function(n) {
  x <- numeric(n)  # Vektor (0, ..., 0) der Länge n
  if (n > 1) {
    x[2] = 1
  }
  if (n > 2) {
    for (i in 2:(n-1)) {
      x[i+1] <- x[i] + x[i - 1]
    }
  }
  x
}
```

- **Beispiel** ($n = 6$; schrittweise Rechnung s. Anhang):

```
fibonacci(6)
```

```
[1] 0 1 1 2 3 5
```

Quiz

Der Algorithmus ist ...

1. exakt ?
2. iterativ ?
3. probabilistisch ?

Algorithmenarten in ML / Statistik

- **Exakt/direkt:**
  - LGS-Lösung via Gauß-Elimination (später...)
  - Matrixmultiplikation
- **Approximativ:**
  - Monte-Carlo-Integration (schätzt Integrale durch Stichproben)
  - Neuronale Netze (optimieren bis zu einem Toleranzlevel)
- **Iterativ:**
  - Gradientenabstieg in Machine Learning
  - Newton-Verfahren zum Nullstellenfinden
- **Probabilistisch:**
  - Approximative Singulärwertzerlegung, Matrix Sketching
  - MCMC-Methoden in Bayesianischer Statistik

Oft kombiniert: ML-Algorithmen sind oft iterativ + approximativ + probabilistisch!

Analogie

Problem:

> “Koche Schweinebraten wie bei Oma.”

Algorithmus $\leftrightarrow$ Kochrezept

Computer $\leftrightarrow$ Koch *(der sich ans Rezept hält, aber Fehler macht)*

Gute Algorithmen

Wir wollen Algorithmen, die

- möglichst exakte Ergebnisse liefern,
- schnell sind,
- wenig Speicher brauchen.

Aufwand und Komplexität

Aufwand und Komplexität

- Der Computer führt einen Algorithmus durch elementare Operationen aus: Rechenoperationen $f_i \in \{+, -, \cdot, /\}$, aber auch Vergleiche, Zuweisungen und Speicherzugriffe — jede mit (etwa) konstantem Aufwand.
- Der *Zeitaufwand* des Algorithmus ist die Anzahl dieser Operationen.
  - höherer Aufwand $\approx$ längere Laufzeit
- Der *Speicheraufwand* ist (in etwa) die Anzahl der (zwischen-)gespeicherten Zahlen.
  - höherer Aufwand $\rightarrow$ mehr Speicherplatz

Beispiel: Matrix-Vektor-Multiplikation

Problem: Berechne $\boldsymbol{y} = \boldsymbol{A}\boldsymbol{x}$ für $\boldsymbol{A} \in \mathbb{R}^{3\times 2}$, $\boldsymbol{x} \in \mathbb{R}^2$.

Konkret: $\boldsymbol{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \\ 5 & 6 \end{pmatrix}$, $\boldsymbol{x} = \begin{pmatrix} 7 \\ 8 \end{pmatrix}$

Berechnung:

$$
\begin{aligned}
y_1 &= 1 \cdot 7 + 2 \cdot 8 = 7 + 16 = 23 \quad (2 \text{ Mult., } 1 \text{ Add.}) \\
y_2 &= 3 \cdot 7 + 4 \cdot 8 = 21 + 32 = 53 \quad (2 \text{ Mult., } 1 \text{ Add.}) \\
y_3 &= 5 \cdot 7 + 6 \cdot 8 = 35 + 48 = 83 \quad (2 \text{ Mult., } 1 \text{ Add.})
\end{aligned}
$$

- **Zeitaufwand**: $3 \times 2 = 6$ Multiplikationen + $3 \times 1 = 3$ Additionen $= 9$ Operationen
- *Allgemein:* Für $\boldsymbol{A} \in \mathbb{R}^{n\times d}$: nd Multiplikationen + $n(d-1)$ Additionen $\approx 2nd$ Operationen
- **Speicheraufwand**: nd (für $\boldsymbol{A}$) + d (für $\boldsymbol{x}$) + n (für $\boldsymbol{y}$) $= nd + n + d$ Zahlen

Quiz: Zeit- und Speicheraufwand

Sei $\boldsymbol{A} \in \mathbb{R}^{n \times d}$, $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^d$,

$$f(\boldsymbol{A}, \boldsymbol{x}, \boldsymbol{y}) = \boldsymbol{A}(\boldsymbol{x} - \boldsymbol{y}) = \sum_{i=1}^{d} (x_i - y_i) \boldsymbol{A}_{\cdot i}.$$

```
f <- rep(0, nrow(A))
for (i in 1:ncol(A)) {
  f <- f + (x[i] - y[i]) * A[, i]
}
```

a) Wie viele elementare Rechenoperationen werden benötigt?

1. $2n + d$
2. $2dn + d$
3. dn^2
4. $2(n + d)$

b) Für wie viele Gleitkommazahlen brauchen wir Speicherplatz?

1. $3n + d$
2. $4nd$
3. $2(n + d)$
4. $n + nd + 2d$

Quiz: Lösung

a) Zeitaufwand: $2nd + d$ Operationen

Aufschlüsselung pro Iteration ($i = 1, \ldots, d$):

1. **Subtraktion:** $(x_i - y_i) \rightarrow$ 1 Operation
2. **Skalierung:** $(x_i - y_i) \cdot \boldsymbol{A}_{\cdot i} \rightarrow n$ Multiplikationen (1 Element/Zeile f. $\boldsymbol{A}_{\cdot i}$)
3. **Addition:** `f <- f + ...` $\rightarrow n$ Additionen

Pro Iteration: $1 + n + n = 2n + 1$ Operationen

Gesamt: $d \times (2n + 1) = 2nd + d$ Operationen

b) Speicheraufwand: $n + nd + 2d$ Gleitkommazahlen

- n (für `f`) + nd (für $\boldsymbol{A}$) + $2d$ (für $\boldsymbol{x}$ und $\boldsymbol{y}$)

Komplexität

- Die **exakte Anzahl** an Operationen und gespeicherten Zahlen zu kennen, ist **nicht wichtig**.
- Wir sind hauptsächlich daran interessiert **wie schnell der Aufwand** mit der Größe des Problems wächst (**“skaliert”**).
- Das nennen wir *(Laufzeit-/Speicher-)Komplexität des Algorithmus*.

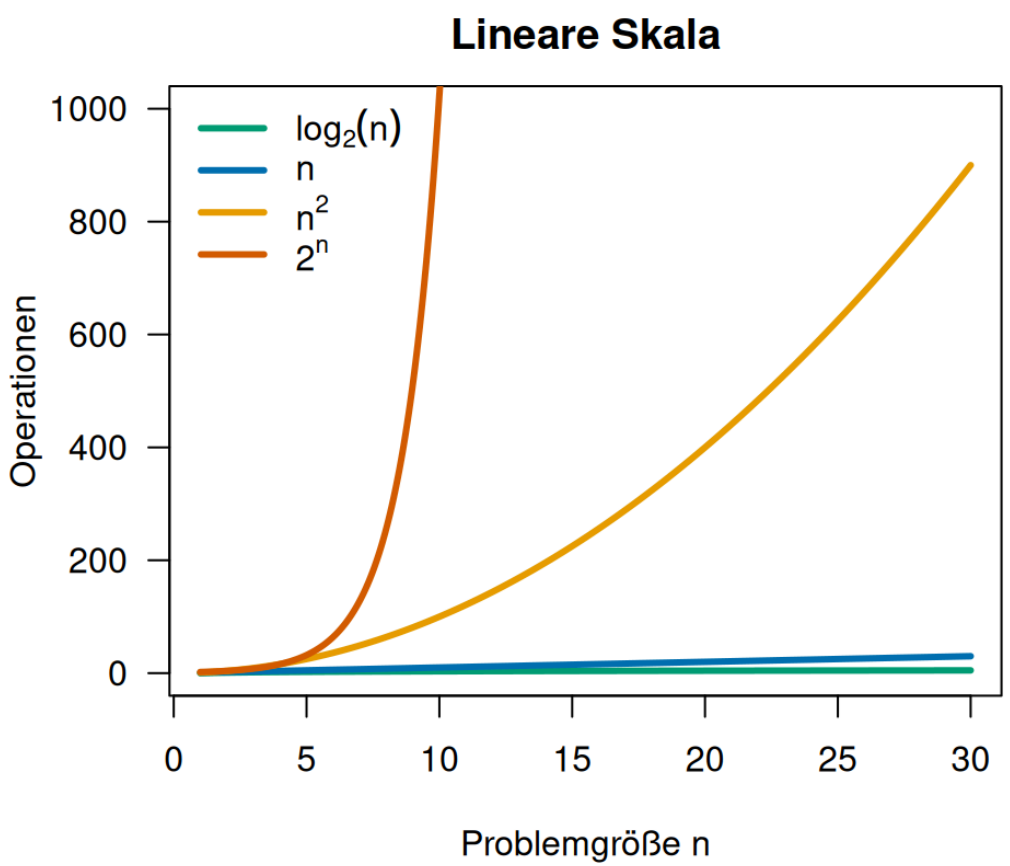
Komplexität

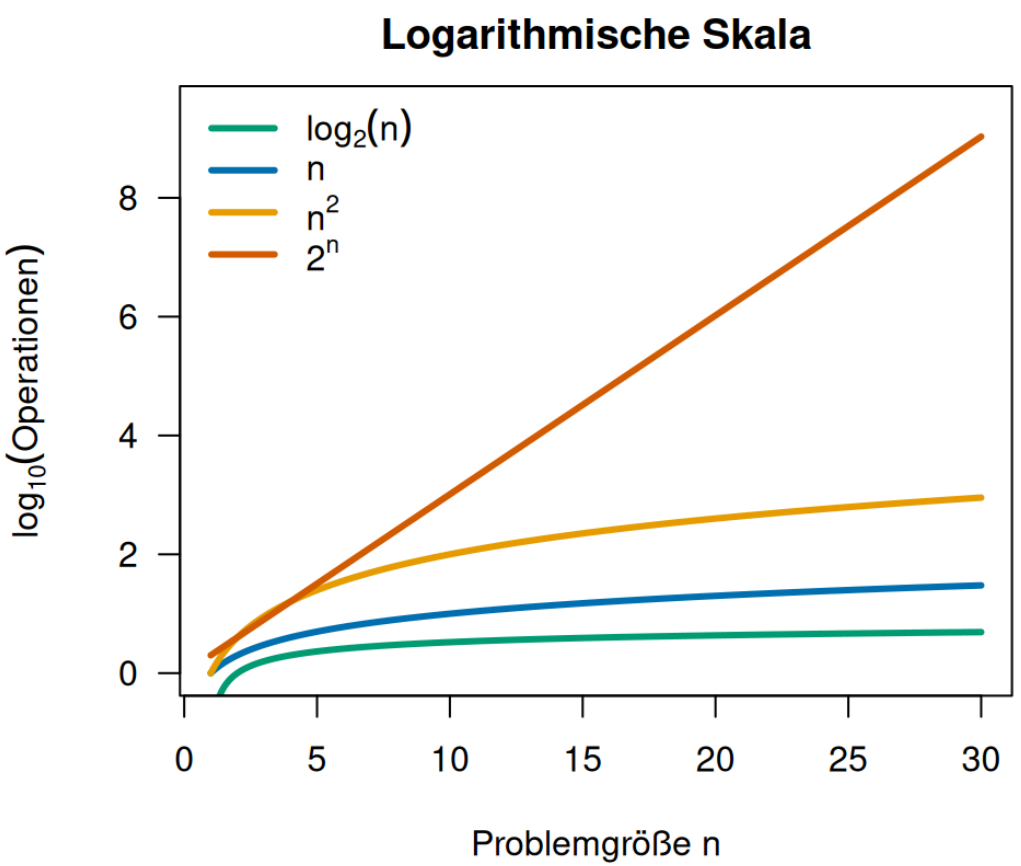
Beispiel:

- Ein Algorithmus benötigt $4n^3 + 16n^2 + 239$ Operationen.
- Für großes n ist $16n^2 + 239$ gegenüber $4n^3$ vernachlässigbar.

$\Longrightarrow$ Der Algorithmus skaliert wie $4n^3$,
die Komplexität ist *kubischer* *Ordnung*: "$O(n^3)$".

Komplexitätsklassen





Komplexitätsklassen

Wie verschiedene Komplexitäten skalieren:

Interpretation:

- $O(1)$ (konstant):
  Anzahl Operationen ist unabhängig von n
- $O(\log_b n)$ (logarithmisch):
  eine Operation mehr falls sich n ver-b-facht.
- $O(n)$ (linear):
  verdoppelt sich n, verdoppeln sich Operationen
- $O(n^2)$ (quadratisch):
  verdoppelt sich n, vervierfachen sich Operationen
- $O(2^n)$ (exponentiell):
  verdoppelt sich n, exponentieren sich Operationen

Vorsicht:

$O()$**-Notation ignoriert konstante Faktoren und Terme** – z.B.

- $1000n + 10000$ Operationen sind immer noch $O(n)$
- aber (viel) langsamer als n^2 Operationen ($O(n^2)$) für $n < 1010$!

Landausymbole

Def.: Landausymbole

Seien a_n, b_n Folgen mit $b_n \neq 0$.

- $a_n = o(b_n)$ („a_n *ist* ***klein-o*** *von* b_n"), wenn

$$\lim_{n\to\infty} \frac{a_n}{b_n} = 0.$$

- $a_n = O(b_n)$ („a_n *ist* ***groß-O*** *von* b_n"), wenn

$$\limsup_{n\to\infty} \left|\frac{a_n}{b_n}\right| < \infty.$$

Interpretation:

- $a_n = o(b_n)$: a_n **wächst langsamer als** b_n, a_n wird gegenüber b_n vernachlässigbar klein.
- $a_n = O(b_n)$: a_n **wächst höchstens so schnell wie** b_n, b_n ist gegenüber a_n nicht vernachlässigbar.
- Zusatz:

$$a_n = \Theta(b_n)$$

falls $a_n = O(b_n)$ und $b_n = O(a_n)$, d.h. a_n **wächst genauso schnell wie** b_n.

Landausymbole: Rechenbeispiel

Beispiel: Zeige, dass $3n^2 + 5n = O(n^2)$

$$\lim_{n\to\infty} \frac{3n^2 + 5n}{n^2} = \lim_{n\to\infty} \left(3 + \frac{5}{n}\right) = 3 < \infty$$
$$\Longrightarrow \; 3n^2 + 5n = O(n^2) \quad \checkmark$$

Weitere Rechenbeispiele: siehe Anhang

Landausymbole

- Meist suchen wir eine möglichst einfache Folge b_n, sodass $a_n = O(b_n)$ aber nicht $a_n = o(b_n)$.
- Schreibweise: $O(b_n)$ und $o(b_n)$ sind *Mengen* von Folgen, genau genommen also $a_n \in O(b_n)$. Das übliche Gleichheitszeichen heißt hier „ist ein" und gilt nur von links nach rechts: aus $n = O(n^2)$ und $n^2 = O(n^2)$ folgt **nicht** $n = n^2$.

Lemma: Ein paar Rechenregeln

Seien $a_n, b_n > 0$ (z.B. Anzahl von Operationen).

1. Sei $f_n = O(a_n)$ und $g_n = O(b_n)$. Dann gilt

$$f_n + g_n = O(a_n + b_n) \quad \text{und} \quad f_n \cdot g_n = O(a_n \cdot b_n).$$

2. Sei $f_n = O(a_n)$ und $g_n = o(b_n)$. Dann gilt

$$f_n \cdot g_n = o(a_n \cdot b_n)$$

3. Sei $f_n = O(a_n + b_n)$ mit $a_n = O(b_n)$. Dann gilt

$$f_n = O(b_n).$$

Rechenregeln: Anwendungsbeispiele

Beispiel Addition und Multiplikation:

Gegeben: $f_n = 3n^2 = O(n^2)$ und $g_n = 5n = O(n)$

$$\text{Addition:} \quad f_n + g_n = 3n^2 + 5n = O(n^2 + n) = O(n^2)$$
$$(\text{wegen Regel 3: } n = O(n^2))$$

$$\text{Multiplikation:} \quad f_n \cdot g_n = (3n^2) \cdot (5n) = 15n^3 = O(n^2 \cdot n) = O(n^3)$$

Beispiel Vereinfachung:

Gegeben: Algorithmus benötigt $4n^3 + 16n^2 + 239$ Operationen.

$$4n^3 + 16n^2 + 239 = O(n^3 + n^2 + 1)$$
$$= O(n^3) \quad (\text{da } n^2 = O(n^3) \text{ und } 1 = O(n^3))$$

$\Longrightarrow$ **Wir können komplexe Ausdrücke auf dominanten Term reduzieren!**

Fibonacci: Komplexitätsanalyse 1

Iterativer Algorithmus:

```r
fibonacci <- function(n) {
  x <- numeric(n)
  if (n > 1) {
    x[2] = 1
  }
  if (n > 2) {
    for (i in 2:(n-1)) {
      x[i+1] <- x[i] + x[i - 1]
    }
  }
  x
}
```

Zeitkomplexität:

- Initialisierung: `x <- numeric(n)` → $O(n)$
- Schleife: $n - 2$ Iterationen mit je konstant vielen Operationen (1 Addition + Indexrechnung) → $O(n)$
- **Gesamt:** $O(n)$ (linear)

Speicherkomplexität:

- Vektor `x` mit n Elementen → $O(n)$

Fibonacci: Komplexitätsanalyse 2

Naive Rekursion:

```
fib_recursive <- function(n) {
  if (n <= 1) {
    return(n)
  }
  return(
    fib_recursive(n-1) +
    fib_recursive(n-2)
  )
}
```

- **Zeitkomplexität:**
  jede Iteration macht 2 Aufrufe, die wiederum je 2 Aufrufe machen, usw.
  $\Longrightarrow$ insgesamt $O(2^n)$, also exponentiell!
  katastrophal langsam
- **Speicherkomplexität:**
  $O(n)$ (Rekursionstiefe)

Live: Laufzeit in Sekunden für $n = 30$ –

```
system.time(fibonacci(30))["elapsed"]
```

```
elapsed
  0.024
```

```
system.time(fib_recursive(30))["elapsed"]
```

```
elapsed
   4.27
```

$\Longrightarrow$ Iterative Lösung ist dramatisch effizienter ($\times 180$) – und liefert dabei sogar *alle* ersten n Zahlen, während die Rekursion nur F_n berechnet!

Self-Check: Landausymbole

Welche Aussagen sind wahr?

1. $2n = O(n^2)$
2. $7/n = o(1)$
3. $8n^3 + 7n^2 + n = O(n)$
4. $n^{-3} + n^{-2} = O(n^{-2})$

Zusammenfassung

- Für numerische *Probleme* f mit Input $\boldsymbol{x}$ wollen wir eine Lösung $f(\boldsymbol{x})$ berechnen.
- Gute *Algorithmen* lösen das *Problem* mit niedrigem Aufwand.
- Wir betrachten deshalb ihre *Laufzeit-/Speicher-Komplexitäten*.
- Diese analysieren wir anhand ihrer *Ordnung* mit *Landausymbolen*.

Nächste Vorlesung

- *Kondition* und *Stabilität*.

Anhang

Fibonacci: Schrittweise Berechnung

Berechne die ersten 6 Fibonacci-Zahlen ($n = 6$):

$$
\begin{aligned}
\widetilde{f}_1(6) &= 0 \\
\widetilde{f}_2(\widetilde{f}_1(6)) = \widetilde{f}_2(0) &= (0, 1) \\
\widetilde{f}_3(0, 1) &= (0, 1, 1) \\
\widetilde{f}_4(0, 1, 1) &= (0, 1, 1, 2) \\
\widetilde{f}_5(0, 1, 1, 2) &= (0, 1, 1, 2, 3) \\
\widetilde{f}_6(0, 1, 1, 2, 3) &= (0, 1, 1, 2, 3, 5)
\end{aligned}
$$

Ergebnis: $\widetilde{f}(6) = (0, 1, 1, 2, 3, 5)$

↩ zurück: „Fibonacci: Implementierung”

Landausymbole: Weitere Rechenbeispiele

Beispiel 2: Zeige, dass $5n = o(n^2)$

$$\lim_{n\to\infty} \frac{5n}{n^2} = \lim_{n\to\infty} \frac{5}{n} = 0 \implies 5n = o(n^2) \quad \checkmark$$

Beispiel 3: Ist $n^2 = O(n)$? **Nein!**

$$\lim_{n\to\infty} \frac{n^2}{n} = \lim_{n\to\infty} n = \infty \quad \text{(divergiert!)}$$
$$\implies n^2 \neq O(n)$$

↩ zurück: „Landausymbole: Rechenbeispiel"